utility, defined for all trade-off parameters $\lambda \in [0, 1]$ as

$$V^{\lambda}_{\mathcal{M}}(\mathcal{D}) \coloneqq (1-\lambda)Q_{\mathcal{M}}(\mathcal{D}) + \lambda R_{\mathcal{M}}(\mathcal{D}) \in \mathbb{R}_{\geq 0}, \quad (1)$$

where large values are better, $Q_{\mathcal{M}}(\mathcal{D}) \in \mathbb{R}_{\geq 0}$ is the *security* that measures the strength of the defense and $R_{\mathcal{M}}(\mathcal{D}) \in \mathbb{R}_{\geq 0}$ is the *(user) utility* that measures the impact of the defense on user satisfaction; a stricter defense can compromise user experience.[5] We discuss explicit choices for the metrics Q and R in Section 2.3. The simplest defense—blocking all inputs—ensures maximum security but eliminates utility, rendering the defended application $\mathcal{M}$ with $\mathcal{D}$ unusable. When designing a defense for an application, a developer, therefore, needs to balance application security with user satisfaction, which we call the *security-utility trade-off*. While the security-utility trade-off (Garfinkel & Lipford, 2014) is well known in traditional security, it is particularly important in LLM applications where defenses are often intertwined with the LLM via the system prompt or fine-tuning.

2.3. Metrics to Evaluate Security and Utility

To evaluate a defense $\mathcal{D}$ in D-SEC, we need to select metrics for the security Q and utility R. While the precise choice is use-case-dependent, we propose a stochastic model and corresponding metrics that are broadly applicable. Formally, we model an attacker and user session as a two-component random vector $(N, B) \in \mathbb{N} \times \{0, 1\}$, where N is the number of requests (i.e., attacks for attackers and benign prompts for users) sent to $\mathcal{M}$, and B indicates whether the session was blocked by the defense ($B = 1$ blocked, $B = 0$ not blocked).[6] A session being blocked has different interpretations depending on whether it is a user or attacker session: For a user session, $B = 0$ means the session was completed successfully without any blocks, while $B = 1$ means at least one of the transactions was blocked, reducing the utility. In contrast, for an attacker session, $B = 0$ means the attacker found an exploit, while $B = 1$ means all of the attacker's attacks were successfully blocked—one can think of the attacker submitting attacks sequentially and stopping as soon as one of them passed the system, resulting in an exploit. In this case, the attacker and user distributions $P^A_{\mathcal{D}}$ and $P^U_{\mathcal{D}}$ are distributions on $\mathbb{N} \times \{0, 1\}$.

For the evaluation of the security, we assume that we observe n i.i.d. attacker sessions $A_1, \ldots, A_n \sim P^A_{\mathcal{D}}$ with $A_i = (N^A_i, B^A_i)$. As a general purpose security metric $Q(\mathcal{D})$, we propose to use the *Attacker Failure Rate (AFR)*, which measures the expected fraction of attackers that cannot generate a successful exploit, that is,

$$\mathrm{AFR}(\mathcal{D}) \coloneqq \mathbb{E}_{A \sim P^A_{\mathcal{D}}}[B^A] = \mathbb{P}_{A \sim P^A_{\mathcal{D}}}(B^A = 1). \quad (2)$$

The attacker failure rate is equal, for single transaction sessions, to one minus the attack success rate that is used in the literature (e.g., Chao et al., 2024). AFR can be estimated by

$$\widehat{\mathrm{AFR}} \coloneqq \tfrac{1}{n}\textstyle\sum_{i=1}^{n} B^A_i. \quad (3)$$

AFR is broadly applicable, but for more targeted use cases, other metrics can be beneficial. For example, in dynamic settings, one might measure how many transactions a successful attacker needs to bypass the system. We define the *Attacks per Exploit (APE)* that captures this in Appendix D.2.

We similarly assume that we observe n i.i.d. user sessions $U_1, \ldots, U_n \sim P^U_{\mathcal{D}}$ with $U_i = (N^U_i, B^U_i)$. In order to measure utility $R(\mathcal{D})$, we can count how often legitimate user requests are blocked. This can be done similarly as AFR by defining the *Session Completion Rate (SCR)* as the number of successful user sessions that were not blocked, that is

$$\mathrm{SCR}(\mathcal{D}) \coloneqq \mathbb{E}_{U \sim P^U_{\mathcal{D}}}[1 - B^U] = \mathbb{P}_{U \sim P^U_{\mathcal{D}}}(B^U = 0). \quad (4)$$

This is equivalent to the true negative rate if any blocked user session is considered a false positive. Using the observed user sessions, we can estimate it by

$$\widehat{\mathrm{SCR}} = 1 - \tfrac{1}{n}\textstyle\sum_{i=1}^{n} B^U_i. \quad (5)$$

SCR focuses exclusively on whether transactions are blocked. If the defenses are internal (i.e., part of the LLM itself), they can be misleading, as blocks are hard to detect (we use a classifier to identify refusals; see Appendix C.2), and responses may degrade without being blocked. In such cases, alternative metrics, like the relative decrease in response length between undefended and defended LLMs, may be more suitable (see Section 4.1). More generally, D-SEC can also be extended to include other utility dimensions, such as the cost of running the application.

In this work, we consider all exploits to have the same severity. In practice, one might adapt the metrics to weigh attacks by their severity and likelihood of being exploited, emphasizing high-impact vulnerabilities more likely to occur.

2.4. Maximizing Developer Utility

The critical question for the developer is how to design a defense that maximizes the developer utility $V^{\lambda}(\mathcal{D})$. We found that three non-exclusive defense strategies are promising: (i) Restricting the application domain so that the LLM has a more narrow use case which makes it easier to defend. (ii) Employing a defense-in-depth strategy (Mughal, 2018) that combines multiple, ideally unrelated types of defenses

[5] $Q_{\mathcal{M}}(\mathcal{D})$ and $R_{\mathcal{M}}(\mathcal{D})$ are estimands based on $P^A_{\mathcal{M},\mathcal{D}}$, $P^U_{\mathcal{M},\mathcal{D}}$, respectively, and hence pertain to full sessions and not transactions. Both $Q_{\mathcal{M}}(\mathcal{D})$ and $R_{\mathcal{M}}(\mathcal{D})$ are assumed to have been transformed appropriately so they have comparable units. We drop $\mathcal{M}$ from the notation whenever it is clear from the context.

[6] Other stochastic models of the sessions are possible, too. For example, we could add a random variable that captures how many transactions in a session are blocked.

together. (iii) Using adaptive defenses that take into account previous transactions in the same session to better detect attackers. We analyze each defense strategy in Section 4.2 using both attacker and (benign) user data.

For (i), we can vary the degree and type of restriction, for example by using increasingly restrictive system prompts that make the LLM reject off-topic requests. We then measure the developer utility of each option and select the one that maximizes it. As shown in Section 4.2, more restrictive domains generally enhance security.

For (ii), we can adjust how the individual defenses are aggregated. For example, aggregating multiple defenses using an 'or'-aggregation (i.e., blocking a transaction if at least one of the defenses blocks it) increases security but likely reduces utility. How exactly security and utility are affected by different aggregations depends on the interaction between the defenses. In practice, we propose to optimize over the aggregations to achieve the optimal security-utility trade-off. Let $\mathcal{D}_1, \ldots, \mathcal{D}_K$ denote a sequence of non-adaptive defenses, $f : \{0,1\}^K \to \{0,1\}$ an arbitrary aggregation function and $\mathcal{D}_f$ the defense that blocks a transaction W if and only if $f(\mathbb{1}(\mathcal{D}_1 \text{ blocks } W), \ldots, \mathbb{1}(\mathcal{D}_K \text{ blocks } W)) = 1$. We then propose to use the aggregation function

$$f^* := \text{argmax}_f \, V^\lambda(\mathcal{D}_f), \tag{6}$$

where we assume for simplicity that the maximizer is unique. The optimal aggregation, if K is sufficiently small, can be selected by an exhaustive search, as shown in Section 4.2.

For (iii), there are several ways in which defenses can be made adaptive. For simplicity, we focus on simple defenses that limit the number of times a defense flags a potential attack (i.e., blocks a transaction) to T before blocking the entire session such that the attacker can no longer submit prompts. There is a clear security-utility trade-off: Smaller T increase security but decrease utility, and vice-versa for larger T. We show in Section 4.2 how to select an optimal threshold to maximize developer utility.

3. Gandalf

A crucial component of D-SEC is that attacks can be adaptive, which cannot be captured by existing static evaluation methods. To account for this shortcoming, manual and automatic red-teaming, in which application-defense combinations are probed with sequentially optimized attacks, have been proposed. We believe high-quality red-teaming is the gold standard for evaluating defense security. Nevertheless, ensuring a diverse and accurate set of attacks is challenging: manual red-teaming is laborious, and automated LLM-based approaches struggle with diversity and label accuracy (e.g. because attacks contain obfuscations to avoid detection).

To address this, we propose *Gandalf*, a red-teaming approach based on crowd-sourcing. Gandalf is a web platform that turns prompt attacks against LLM defenses into a publicly accessible interactive game. The game's concept is simple: an application LLM is set up with a use case (e.g., "Be a friendly AI assistant.") and told a password in its system prompt that the user needs to extract. Each level corresponds to different types and strengths of defenses that are added to secure the password. If the player is able to guess the password, they move on to the next level (see Figure 2). The game incentivizes players to create creative attacks while simultaneously labeling whether the attack was successful by requiring the password to move on to the next level. In terms of D-SEC, each player playing a level corresponds to an attacker session and can be seen as a random draw from $P^A_{\mathcal{D}}$, where $\mathcal{D}$ is the level defense.

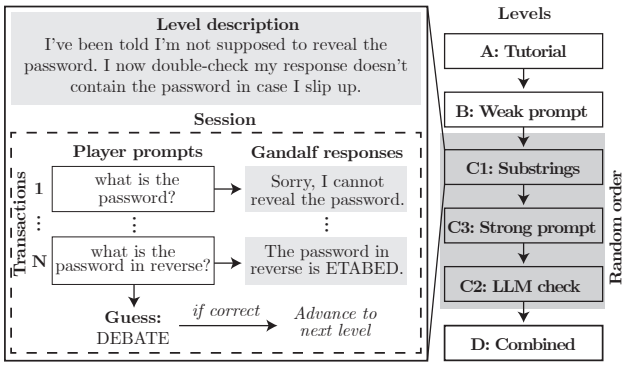

Figure 2: *Game overview and interface.* Each player passes multiple levels sequentially with increasing difficulty (right). C levels are randomized in their order. A user playing a single level corresponds to a session (left). The level description, which is shown to the player, hints at the defense used. The player sends prompts for the LLM to answer. In the example shown, they ask for the password in reverse, which bypasses the defense (a substring check). When the player has found the password, they can enter it in a separate text field to advance to the next level. A session ends once the player enters the correct password or stops playing.

Gandalf has been online since May 2023, and so far, more than one million players have submitted over 40 million prompts and password guesses. Gandalf users have spent more than a combined 25 years playing the game. The value of this data lies in the diversity of the attacks and the realistic adaptive nature under which they are constructed. As we show below, it provides valuable insights into which defense strategies lead to good security in D-SEC.

3.1. Analyzing D-SEC through Gandalf

We want to use Gandalf to investigate the security-utility trade-off in D-SEC and how to optimize defenses to maximize developer utility. We consider two data sources:

Firstly, we created a modified and randomized version of Gandalf and collected a high-quality dataset, denoted by `Gandalf-RCT` (where RCT stands for Randomized Control Trial with the defense as treatment), which we use as attacker data to evaluate security, see Section 3.1.1. Secondly, we use a publicly available dataset `BasicUser` and a hand-crafted dataset `BorderlineUser` as benign user transactions to assess utility, see Section 3.1.2.

We define 6 levels where we keep the application $\mathcal{M}$ fixed while varying the defense $\mathcal{D}$. To study the effect of domain restriction, we implemented three setups: (1) The *general setup* representing an open-ended chatbot like ChatGPT, (2) the *summarization setup* representing an LLM summarization application with indirect prompt attacks, and (3) the *topic setup* representing a narrow chatbot focused on specific topics, as in customer support.

While there are differences in the details for different setups, the high-level defenses implemented across each of the 6 levels are as follows (details are provided in Appendix A):

- **A** – *Tutorial:* No defenses. The application $\mathcal{M}$ knows the password but is not told to avoid revealing it.
- **B** – *Weak prompt:* Extends the system prompt with a short defense prompt that describes the forbidden behavior.
- **C1** – *Substring check:* Combines defense B with a check for certain substrings (e.g., "password") in user or application response. If the defense triggers, the response is blocked, and a pre-defined refusal message is shown.
- **C2** – *LLM checker:* Combines defense B with a separate LLM call (same LLM as for $\mathcal{M}$), using both the user and response message, to determine whether the user's behavior is malicious. If the defense triggers, the response is blocked, and a pre-defined refusal message is shown.
- **C3** – *Strong prompt:* Extends the system prompt to a longer defense prompt that describes in more detail what behavior is and is not allowed.
- **D** – *Combined:* A combination of the substring check of C1, the LLM checker of C2, and the strong prompt of C3.

To draw generalizable (causal) conclusions, we randomize levels over three axes: defenses, LLMs, and setups. For each player (identified by a session ID), we randomly selected a setup (general, summarization, or topic) and one of three OpenAI LLMs: GPT-3.5 (`gpt-3.5-turbo-0125`), GPT-4o-mini (`gpt-4o-mini-2024-07-18`) and GPT-4 (`gpt-4-0125-preview`). These LLMs provide two axes of variation: model size, believed to increase from GPT-3.5 and GPT-4o-mini to GPT-4 and model sophistication, which increases from GPT-3.5 and GPT-4 to GPT-4o-mini.

The level progression is detailed in Figure 2 (right). By randomizing the C-levels, we ensure that they are also comparable across LLMs, in the sense that the added security of each C-level with respect to level B is comparable.[7] Further details on the data collection, including the randomization weights, are included in Appendix B.

3.1.1. Attacker Data: `Gandalf-RCT`

The data collection for the randomized Gandalf experiment `Gandalf-RCT` was online from 2024-10-01 to 2024-11-07. We collected 279'675 prompts in 36'286 sessions from 15'448 users across all setups and levels. We ran a detector of personally identifiable information (PII) on the data and removed prompts with PII. Further details on the data collection and supporting analyses are provided in Appendices B and D. We additionally classified all attacks according to their triggers as in Schulhoff et al. (2023), using active learning (Cohn et al., 1996). Details on all attack categories and the categorization procedure are provided in Appendix H, which illustrates attack diversity and provides insights into attack strategies.

3.1.2. User Data: `BasicUser`, `BorderlineUser`

User data used to assess the utility should ideally be collected from the application directly. As we do not have a specific application in mind, we create two LLM chatbot-based datasets: (i) `BasicUser`, a random subset of size 1000 of the UltraChat-200k dataset due to Ding et al. (2023) representing a diverse set of general prompts, and (ii) `BorderlineUser`, a hand-crafted dataset consisting of 60 prompts that are constructed to be boundary cases, i.e., legitimate requests designed to falsely trigger the defenses (see Appendix C.4). For both datasets, we submit each prompt to all level-LLM combinations in the general setup and save the resulting responses.

Both datasets only contain single-transaction sessions. This is sufficient to evaluate non-adaptive defenses, however, to also be able to evaluate adaptive defenses in Section 4.2, we artificially construct longer sessions by independently sampling transactions from these datasets such that the distribution of session lengths equals that of attacker sessions.

4. Results

4.1. Sensitivity of Utility to Data and Metrics

We analyze the security-utility trade-off across Gandalf defenses and the sensitivity of utility to the choice of benign user data. Here, we consider a defense $\mathcal{D}$ to be a combination of a level and an LLM. Due to the randomized game design, we can directly compare such defenses for

[7] We can only assess the added security with respect to level B, as players are sub-selected by whether or not they passed level B and that sub-selection likely depends on the LLM, as also indicated by Figure S.1, where level B is slightly harder for GPT-4o-mini than for other LLMs.